4×4 Matrix Feedback Generator

Implementation Plan — 16 sequenced steps

This plan breaks the full plugin build into 16 sequenced, independently-scoped steps. No code is written in this document — each step below is a self-contained brief (Description + Prompt) meant to be handed to a future coding session to execute just that step. Every prompt references `4x4-System-Overview.md` (the authoritative spec for every locked decision) and, where relevant, `CONTOUR_NT_SPEC.md` / `contour_engine.h`. Work happens in `/4x4/4x4.cpp` per this project's plugin-folder convention (`CLAUDE.md`) — move superseded versions into `/4x4/History/` before saving a new one, once versions start accumulating.

> **What "finished" means here:** a plugin that compiles, loads, and plays back kits matching every decision currently locked in `4x4-System-Overview.md`. It does **not** mean every open question in that doc's §10 is resolved — items explicitly marked "not blocking" (the eventual sliced/polyphonic sampler beyond the interim Play-loop player, kit-swap musical continuity, exact AGC/pitch-latency tuning) are deliberately out of scope for this plan and remain future work after Step 16.

Dependency overview

#	Step	Depends on	Type
1	Project scaffolding and build verification	—	Code
2	Hand-written FFT primitive	1	Code
3	Lazy Contour engine reimplementation	—	Code
4	P1 — pitch shifter processor	2	Code
5	P4 — bitcrusher / sample-rate-reducer processor	1	Code
6	P2 — source/derive allpass coefficients	—	Research
7	P2 — frequency shifter processor	6	Code
8	P3 — derive exact geometric fold formula	—	Design
9	P3 — wavefolder processor	8	Code
10	Feedback matrix, fixed injection points, starter wave	4, 5, 7, 9	Code
11	HPF/LPF filters and Reg (regulator)	10	Code
12	Contour instance management and destination wiring	3, 4, 5, 7, 9, 11	Code
13	Kit buffer and double buffering	—	Code
14	Renderer state machine	10, 11, 12, 13	Code
15	Player and full parameter/factory integration	13, 14	Code
16	On-device build, load, and profiling verification	15	Verification

Steps 3 and 13 have no hard dependency on the processor steps and can be built in parallel if working across sessions concurrently.

Step 1 — Project scaffolding and build verification

Depends on: nothing

Before any DSP code, confirm the toolchain works end to end: a minimal algorithm that compiles with the exact flags from `CLAUDE.md`, produces a valid relocatable `.o`, and does nothing except output silence. Exists purely to catch build/toolchain/linkage problems before they're buried under real DSP code.

> *Set up the 4x4 plugin's project scaffolding per this project's CLAUDE.md conventions. Create /4x4/4x4.cpp with a minimal but complete disting NT plugin: factory struct, pluginEntry(), calculateRequirements(), construct(), and a step() that does nothing (outputs silence). No parameters beyond what's structurally required to compile. Compile with the exact flags from CLAUDE.md's "Compiler Flags" section and confirm a clean .o is produced. This step validates the build pipeline only — no real DSP, no parameters from 4x4-System-Overview.md yet.*

Step 2 — Hand-written FFT primitive

Depends on: Step 1

P1 needs an in-place, float32 FFT/IFFT since the disting NT API provides none (§2). Substantial enough to build and validate as a standalone, portable C++ unit before wiring it into pitch-shift-specific logic.

> *Implement a standalone, in-place, radix-2 FFT/IFFT in portable float32 C++ (<cmath> only, no STL, no dynamic allocation), sized for 2048-point complex transforms per 4x4-System-Overview.md §2 (1024-sample analysis block, 2× zero-padded to 2048). Write it as an isolated header/source pair, not yet wired into any plugin. Validate on desktop (standard g++/clang, flags minus the ARM-specific ones, matching the testing approach CONTOUR_NT_SPEC.md §8.1 uses for Contour) against a known reference FFT or hand-computed test vectors (DC, single sine, impulse). Do not implement pitch-shift logic yet — FFT primitive only.*

Step 3 — Lazy Contour engine reimplementation

Depends on: nothing (can run in parallel with Steps 1–2)

The single biggest architectural commitment in this design — replacing Contour's baked breakpoint-storage engine with a lazy, on-demand evaluator (§6/§8). Explicitly deviates from CONTOUR_NT_SPEC.md's "use the reference engine as given," so needs validation against that spec's golden test values, not just a memory win. The §8 sizeof() estimate (≈1.0KB/instance) is flagged as unmeasured — this step's definition of done includes measuring it for real.

> *Reimplement contour::Engine as a lazy, on-demand evaluator per 4x4-System-Overview.md §6/§8: only the current envelope segment's scalars plus one segment of lookahead (needed for corner-rounding, which blends using the next segment's slope) are retained in memory, computed via Contour's keyed random lookup (rnd(seed,i,j)) the moment playback reaches each segment — not precomputed and stored as breakpoint arrays like the reference engine's rebuild(). The hold/round region Span lists still need eager precomputation (a genuine whole-timeline decision, per the doc). Validate against CONTOUR_NT_SPEC.md §8's golden test cases (exact sample values at specific times, within ±0.001V) to confirm equivalent output to the reference engine, not just a smaller one. Measure the actual sizeof() per instance at native settings (kMaxWin=32, kPieces=24) and update 4x4-System-Overview.md §8 with the real number, replacing the ≈1.0KB design-time estimate — if it differs meaningfully, flag the knock-on effect on the locked ≈4.90s kit duration. Standalone and tested here — not yet wired into the plugin's destinations.*

Step 4 — P1: pitch shifter processor

Depends on: Step 2

The first full processor: Ocean's algorithm per §2, using the FFT from Step 2, including the newly-locked base-randomization behavior. Since "seed-derived vs. true entropy" for that randomization is still explicitly open (§10), implement it switchably rather than hardcoding one approach.

> *Implement P1 (pitch shifter) per 4x4-System-Overview.md §2, using the FFT from Step 2: 1024-sample Hann-windowed analysis block, hop 256 (4× overlap), 2× zero-padded spectrum, unity-root bin-shift complex multiplication, overlap-add resynthesis. In-place, float32 throughout. Standalone processor unit (input/output sample arrays + persistent state struct), not yet wired into the full plugin. Base value: randomize fresh at the start of each render within ±24 semitones, held fixed for that render — put the random draw behind a small interface so it can be swapped between seed-derived and hardware-entropy sourcing once §10's open question is resolved; don't hardcode a choice. Depth is a plain scalar input for now (becomes a real parameter in Step 15). Validate on desktop: confirm a known input frequency shifts by the expected ratio for a few test semitone values.*

Step 5 — P4: bitcrusher / sample-rate-reducer processor

Depends on: Step 1

The simplest of the four processors — no FFT, no filter coefficients, trivial state. Good to build early to validate the "processor interface" pattern cheaply before the harder P2/P3.

> *Implement P4 (bitcrusher/sample-rate-reducer) per 4x4-System-Overview.md §5: zero-order-hold sample-rate reduction (linear integer divisor 1–64, referenced to the render's own internal processing rate, not the kit's 32kHz storage rate) feeding a round-to-nearest-step bit-depth quantizer (integer-stepped 1–16 bits, no dither). Standalone processor unit taking bit-depth and rate-divisor as plain scalar inputs (their Contour-curve wiring is Step 12's job). No anti-aliasing filter — aliasing is intentional per the doc. Validate on desktop with a sine input at a few bit-depth/divisor combinations, confirming the expected stepped/held character by inspecting output samples.*

Step 6 — P2: source or derive allpass coefficients

Depends on: nothing (research/design step, not coding)

§3/§10 is explicit that the allpass coefficient set must come from a validated filter-design reference or be properly derived — "not fabricated." Must happen before Step 7 can produce real code rather than placeholder guesses.

> *Source or derive the IIR allpass-pair Hilbert transformer coefficients for P2, per 4x4-System-Overview.md §3: two parallel chains of allpass sections (first/second-order) whose phase difference stays close to 90° across the audio band. This must come from a validated filter-design reference or be properly derived via a filter-design method — not fabricated or guessed. Document the chosen section count, the coefficients themselves, and the frequency range over which the 90° phase relationship actually holds (allpass Hilbert approximations typically degrade at the low end — state the usable low-frequency bound explicitly). Update 4x4-System-Overview.md §3/§10 to mark this gap resolved before Step 7 begins.*

Step 7 — P2: frequency shifter processor

Depends on: Step 6

The actual P2 implementation, unblocked by Step 6. Also needs the parameter spec (unit/range, Contour-destination treatment) that §3/§10 flag as undefined — settle that here, same as P1's was settled in §2.

> *Implement P2 (frequency shifter) per 4x4-System-Overview.md §3, using the coefficients from Step 6: two allpass chains → rotating-phasor complex multiply (cos/sin at the shift frequency) → real-part extraction. Recursive, sample-by-sample, no block latency. Standalone processor unit. Also settle P2's parameter spec, which §3/§10 leave open: decide the shift unit and range (e.g. Hz, bipolar), and whether it's curve-only (like P4) or base+depth (like P1 — in which case base must be randomized per render, same rule as P1, not performer-set). Update 4x4-System-Overview.md §3/§9/§10 with the decision. Validate on desktop: confirm a test tone's spectral peak moves by the expected fixed Hz amount (not a ratio) at a few shift settings.*

Step 8 — P3: derive exact geometric fold formula

Depends on: nothing (design step, not coding)

§4/§10 locks the category (geometric mirror fold, mod+abs construction) but not the precise formula — corner placement, period, and how fold/bias numerically map onto it are still undefined.

> *Derive the exact geometric "mirror" fold formula for P3 per 4x4-System-Overview.md §4: a mod + absolute-value construction producing hard-cornered reflections (not a smooth sine-fold). Specify precisely: the period of the folding function, where the reflection corners sit, and the exact numeric mapping from the fold and bias parameters onto the formula's shape (y = foldShape(x·fold + bias), per the doc — define foldShape() exactly). Sanity-check the formula stays within a bounded output range for any input (the doc relies on this self-bounding property, §4). Update 4x4-System-Overview.md §4/§10 with the finalized formula before Step 9 begins.*

Step 9 — P3: wavefolder processor

Depends on: Step 8

The actual P3 implementation. Also needs the parameter spec (unit/range for fold and bias, Contour-destination treatment) that §4/§10 flag as open — settle this here, same as Step 7 did for P2.

> *Implement P3 (wavefolder) per 4x4-System-Overview.md §4, using the formula from Step 8: closed-form evaluation (no lookup table), fold and bias applied as pre-scale/pre-offset before the fixed nonlinearity. Standalone processor unit. Settle P3's parameter spec, which §4/§10 leave open: unit/range for fold and bias, and whether either is curve-only or base+depth (if base+depth, base is randomized per render, same rule as P1/P2). Update 4x4-System-Overview.md §4/§9/§10 with the decision. Validate on desktop: plot or print the transfer curve for a few fold/bias settings and confirm it matches the mirror-fold shape in §4's Fig. 4.*

Step 10 — Feedback matrix, fixed injection points, and starter wave generator

Depends on: Steps 4, 5, 7, 9

The network plumbing tying the four processors together: the 16-crosspoint bipolar feedback mixer, the four fixed (hard-wired) injection points, and the starter-wave square-wave generator that feeds them once per render. This is where Pn stops being four isolated units and becomes an actual feedback network.

> *Implement per 4x4-System-Overview.md §1: the 4×4 bipolar feedback mixer (16 crosspoints, every processor output routed to every processor input), the four fixed injection points (direct, hard-wired Sn→Σn, no gain stage), and the starter-wave square-wave generator ("Starter wave injection" in §1): a phase accumulator per channel, 4 independent frequency and 4 independent duration parameters (0.1–0.5s), firing once as a burst at the start of each render's virtual timeline. Wire Σn = fixed injection + feedback mix for each of the 4 channels, using the processors from Steps 4/5/7/9. Do not include HPF/LPF or Reg yet (Step 11) — Pn's raw output feeds directly back into the matrix for now, keeping this step's scope to the matrix/injection topology alone.*

Step 11 — HPF/LPF filters and Reg (regulator)

Depends on: Step 10

Completes the per-channel chain: Σn → HPF → LPF → Pn → Reg → Out n. Reg's AGC target is locked (§1); peak-vs-RMS detection is explicitly still open (§10) and worth a listening decision rather than a silent default.

> *Implement per 4x4-System-Overview.md §1: a simple HPF→LPF pair per channel (cutoffs will be Contour-driven later, Step 12 — take them as plain scalar inputs for now), and the Reg regulator chain: DC block → AGC → soft clip (tanh). AGC target is locked: −18dB to −6dB relative to the hardware's ±11V ceiling (≈1.39V–5.51V peak), attack ~2–10ms, release ~100–500ms. The doc leaves peak-vs-RMS level detection explicitly open — implement one (peak detection is the simpler default) but flag clearly in code comments and in 4x4-System-Overview.md §10 that this needs a listening pass to confirm, not just a default pick. Wire Reg's output to split to Out n and back into the feedback mixer's column n, completing Step 10's matrix. Validate on desktop: feed a test signal through the full per-channel chain and confirm the AGC settles output level into the target band over a few hundred ms.*

Step 12 — Contour instance management and destination wiring

Depends on: Step 3, Steps 4/5/7/9/11

Ties the lazy Contour engine to every modulation destination now that they all exist: 16 crosspoints, 8 cutoffs, P1 depth, P4's two parameters, and whatever P2/P3 settled on. Implements the shared-settings-bank + per-instance-seed pattern, the output-unit mapping layer (still flagged open in §6/§10), and Length locked to kit duration. This is the step where the final, real destination count exists — §10 has been waiting on this since P2/P3 were undefined.

> *Implement the Contour instance-management layer per 4x4-System-Overview.md §6: one lazy contour::Engine instance (Step 3) per modulation destination, sharing one global settings bank (Windows, Chaos, Radius, Round prob, Hold prob — Length excluded, locked to equal the kit's maximum duration) with only the derived seed (base seed + destination index) varying per instance. Enumerate every real destination from Steps 4/5/7/9/11: 16 feedback crosspoints, 8 filter cutoffs, P1 depth, P4 bit-depth, P4 rate-divisor, plus whatever P2/P3 landed on. Implement the output-unit mapping layer the doc flags as undesigned (§6/§10): Contour's linear-volt output needs converting to each destination's native scale — direct for crosspoint-gain-type destinations (via Level/Offset/Bipolar), logarithmic/exponential for Hz-type destinations like filter cutoffs. Update 4x4-System-Overview.md §8's memory table with the final, real destination count and recompute the kit-duration figure accordingly — report the new number clearly, since it will differ from the ≈4.90s estimate that assumed only 27.*

Step 13 — Kit buffer and double buffering

Depends on: nothing structurally (can run in parallel with Steps 10–12; sequenced here since Step 14 needs it)

The storage layer — 12-bit packed/32kHz mono format, double-buffered, atomic swap on completion. Mechanical but precision-sensitive: getting the 12-bit packing exactly right matters for both correctness and the locked memory budget.

> *Implement the kit buffer per 4x4-System-Overview.md §8: 12-bit packed, 32kHz, mono audio storage, double-buffered (current kit + next kit, both fully resident). Implement the 12-bit pack/unpack routines precisely (define the exact bit-packing scheme, e.g. two 12-bit samples per 3 bytes) and verify round-trip accuracy against float32 test signals (confirm quantization error is in the expected ~12-bit range). Implement the atomic pointer-swap mechanism: the renderer (Step 14) only ever writes the inactive buffer, and the swap to make it current happens only once a render is fully complete — the player must never observe a partially-rendered buffer. Size the buffer using the real duration figure from Step 12's recomputation, not the earlier ≈4.90s placeholder.*

Step 14 — Renderer state machine

Depends on: Steps 10, 11, 12, 13

The throttled, chunked background computation that actually produces kit audio — advancing the full signal chain by one bounded chunk per step() call, tracking its own virtual sample-position counter, writing into the Step 13 kit buffer. Also where "state continuity between renders" (§10, still open) gets decided: does processor/Reg internal state carry across kit generations or reset cold.

> *Implement the renderer state machine per 4x4-System-Overview.md §6/§7: an explicit, resumable state machine (not a single long pass) that advances the full signal chain from Steps 10–12 by one bounded chunk per step() call, maintaining its own virtual sample-position counter independent of wall-clock time or step() call count. All Contour-driven destinations and the starter-wave burst are evaluated against this virtual position, not real time. Pick an initial chunk size, document the reasoning, and leave it easily tunable — render speed trades directly against peak per-call CPU and there's no fixed target yet (needs on-device profiling, Step 16). Decide and implement whether processor-internal state (P2's allpass memory, Reg's DC-block/AGC envelope state) carries over between kit generations or resets cold at each new render — §10 leaves this open; pick one, document the reasoning in 4x4-System-Overview.md §10, and mark it resolved. Write completed audio into the kit buffer's inactive slot (Step 13), triggering the atomic swap only when a render finishes. Re-trigger (not reconstruct) every Contour instance at the start of each new kit generation, per §6, so envelope levels carry forward rather than resetting cold.*

Step 15 — Player and full parameter/factory integration

Depends on: Steps 13, 14, and effectively everything prior

Wires everything into the actual _NT_algorithm/_NT_factory structure: the interim Play/loop player (§6), the Output bus parameter, and the complete parameters[] array. This is the step that produces an actually-loadable, actually-playable plugin for the first time.

> *Implement the player and full plugin integration per 4x4-System-Overview.md §6/§9: the interim Play parameter (on/off — loops the current kit buffer from Step 13 continuously when 1, outputs silence when 0) and the Output bus parameter (standard disting NT bus-select pattern; confirm during this step whether a paired Output Mode parameter is needed per the API's _WITH_MODE convention, per §10's open question, and implement accordingly). Assemble the complete parameters[] array: starter frequencies/durations ×8, P1 depth, Play, Output bus, whatever P2/P3 settled on in Steps 7/9, and the shared Contour settings bank (confirm during this step whether that bank is real parameters or compiled-in constants, per §9's still-open question, and implement accordingly). Wire calculateRequirements()/construct()/parameterChanged()/step() per the standard API lifecycle, placing the renderer (Step 14) and kit buffer (Step 13) in sram/dram/dtc as sized in §8. Follow this project's plugin-versioning convention: this produces the first complete 4x4.cpp/4x4.o in /4x4/.*

Step 16 — On-device build, load, and profiling verification

Depends on: Step 15

The reality check. Compile for the actual ARM target, confirm it loads on the module, and profile against every locked-but-estimated number in 4x4-System-Overview.md: memory usage vs. the §8 budget, CPU per render chunk vs. the §7 peak-per-call constraint, and kit-bake wall-clock time (deliberately left unfixed, §7 — now it gets a real number).

> *Build 4x4.cpp for the ARM target with the exact flags from CLAUDE.md, copy the .o to /programs/plug-ins/ on the module's MicroSD, and confirm it loads cleanly (Misc / Plug-ins / View info… per CLAUDE.md). Profile and report: actual DTC/DRAM usage against the §8 budget (confirm the real Contour sizeof() from Step 3 and the real destination count from Step 12 actually fit); actual CPU percentage per step() call across a range of render-chunk sizes, to find a chunk size that stays safely under the module's overload threshold with headroom for other algorithms (§7's LASER 20–30%-CPU reference point is the rough ceiling to beat); actual wall-clock time to bake one full kit at the chosen chunk size. Update 4x4-System-Overview.md throughout with these real numbers, replacing every design-time estimate this plan deferred to this step.*